Résolution approchée d'équations différentielles

MPSI 2-3
CPGE Oujda

A.MAZZA

mazza8azzouz@gmail.com

EDO du 1er ordre

Principe général

Soit $n \in \mathbb{N}^*$ et U un ouvert de $\mathbb{R}x\mathbb{R}$. Une **équation différentielle d'ordre 1** (résolue) est une équation de la forme :

$$y'(t) = F(y(t), t)$$

où F est une application continue sur U à valeurs dans $\mathbb{R}^n$ et où la fonction inconnue y est de classe $C1$ sur un certain intervalle de $\mathbb{R}$, à valeurs dans $\mathbb{R}^n$.
Dans le cas où $\mathbf{n = 1}$, on parle d'une équation différentielle scalaire ; dans le cas général, on obtient un système différentiel.
Si $(t0, y0) \in U$, résoudre le problème de Cauchy en $(t0, y0)$, c'est trouver un couple (I, y) où I est un intervalle de R et où y est une fonction de classe $C1$ de I dans $\mathbb{R}^n$ tels que

$$y(t0) = y0 \ et \ \forall t \in I y'(t) = F(y(t), t)$$

Méthode d'Euler

La méthode d'Euler : Principe

- On cherche à résoudre sur [a, b] l'équation de la forme :

$$y'(t) = F(y(t), t) \; avec \; y(t0) = y0$$

- Si $y(t)$ est de classe $C1$ alors, connaissant $y(t_i)$, on peut évaluer de façon approché, $y(t_{i+1})$ par son développement de Taylor à l'ordre 1 :

$$y(t_{i+1}) = y(t_i) + y'(t_i) x (t_{i+1} - t_i)$$

- Soit en posant $h = t_{i+1} - t_i$ et avec $y'(t_i) = f(y(t_i), t_i)$:

$$y(t_{i+1}) = y(t_i) + f(y(t_i), t_i) x h$$

- Connaissant $y(a)$, on évalue par itérations successives les valeurs approchées de la fonction $y(t)$:

$$[y(a), y(a+h), y(a+2h), y(a+3h) ... y(b-h), y(b)]$$

Équation différentielle : Exemple 1

Soit l'EDO :

$$\frac{dy}{dt} = ay(t) \quad \text{avec} \quad y(0) = 1$$

Mise sous forme générique :

$$\frac{dy}{dt} = f(y, t)$$

Paramètres et itération

Paramètres :

- fonction $f(y, t)$
- condition initiale y_0
- temps final t_f
- pas de temps dt

Itération à $t^{n+1} = t^n + dt$:

$$y^{n+1} = y^n + dt \cdot f(y^n, t^n)$$

Analyse de la précision

- Calcul de la solution pour différents pas $dt = 1, 0.1, 0.01$
- Calcul de l'erreur avec la solution exacte $y_e(t) = e^{at}$
- Étude du comportement pour $a = 0.2,\ 1,\ 2$

Fonction générique Euler (structure)

Écriture d'une fonction générique dépendant des paramètres :

```
def Euler(f, y0, tf, dt):
    """ Intgration de dy/dt = f(y, t) par Euler
        avec une CI y0 pendant tf avec un pas dt """
    n = int(tf / dt)
    T = np.zeros(n + 1)
    Y = np.zeros(n + 1)
    Y[0] = y0
    T[0] = 0.0
    for i in range(n):
        Y[i+1] = Y[i] + dt * f(Y[i], T[i])
        T[i+1] = T[i] + dt
    return Y, T
```

Résumé des paramètres et itération

Paramètres :

- fonction $f(y,t)$
- condition initiale y_0
- temps final t_f
- pas de temps dt

Itération :

$$t^{n+1} = t^n + dt \quad , \quad y^{n+1} = y^n + dt \cdot f(y^n, t^n)$$

Analyse de la précision (rappel)

- Étude pour différents $dt = 1, 0.1, 0.01$
- Comparaison avec $y_e(t) = e^{at}$
- Comportement selon $a = 0.2, 1, 2$

Second membre : fonction $F(y,t)$

Définition de l'équation différentielle :

$$\frac{dy}{dt} = ay(t) \quad \text{avec} \quad y(0) = 1$$

Définition de la fonction Python :

```
# second membre
def F(y, t):
    """2nd membre de l'EDO"""
    global a
    return a * y
```

Traçage de la solution et de l'erreur

Fonction Python pour tracer solution et erreur :

```
def tracesol(T1, Y1, T2, Y2, T3, Y3, titre):
    """Trace solution et erreur"""
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.title(titre)
    plt.plot(T1, Y1, '<-', label="dt=1")
    plt.plot(T2, Y2, '^-', label="dt=0.1")
    plt.plot(T3, Y3, '+-', label="dt=0.01")
    plt.legend()
    plt.subplot(1, 2, 2)
    plt.title("erreur␣relative␣|(uex-uh)/uex|")
    plt.semilogy(T1, np.abs(Y1 - np.exp(a * T1)) / np.
        exp(a * T1), label="Err1")
    plt.semilogy(T2, np.abs(Y2 - np.exp(a * T2)) / np.
        exp(a * T2), label="Err2")
    plt.semilogy(T3, np.abs(Y3 - np.exp(a * T3)) / np.
        exp(a * T3), label="Err3")
```

Application : Résolution numérique

Équation :

$$\frac{dy}{dt} = ay(t) \quad \text{avec} \quad y(0) = 1$$

Objectif :

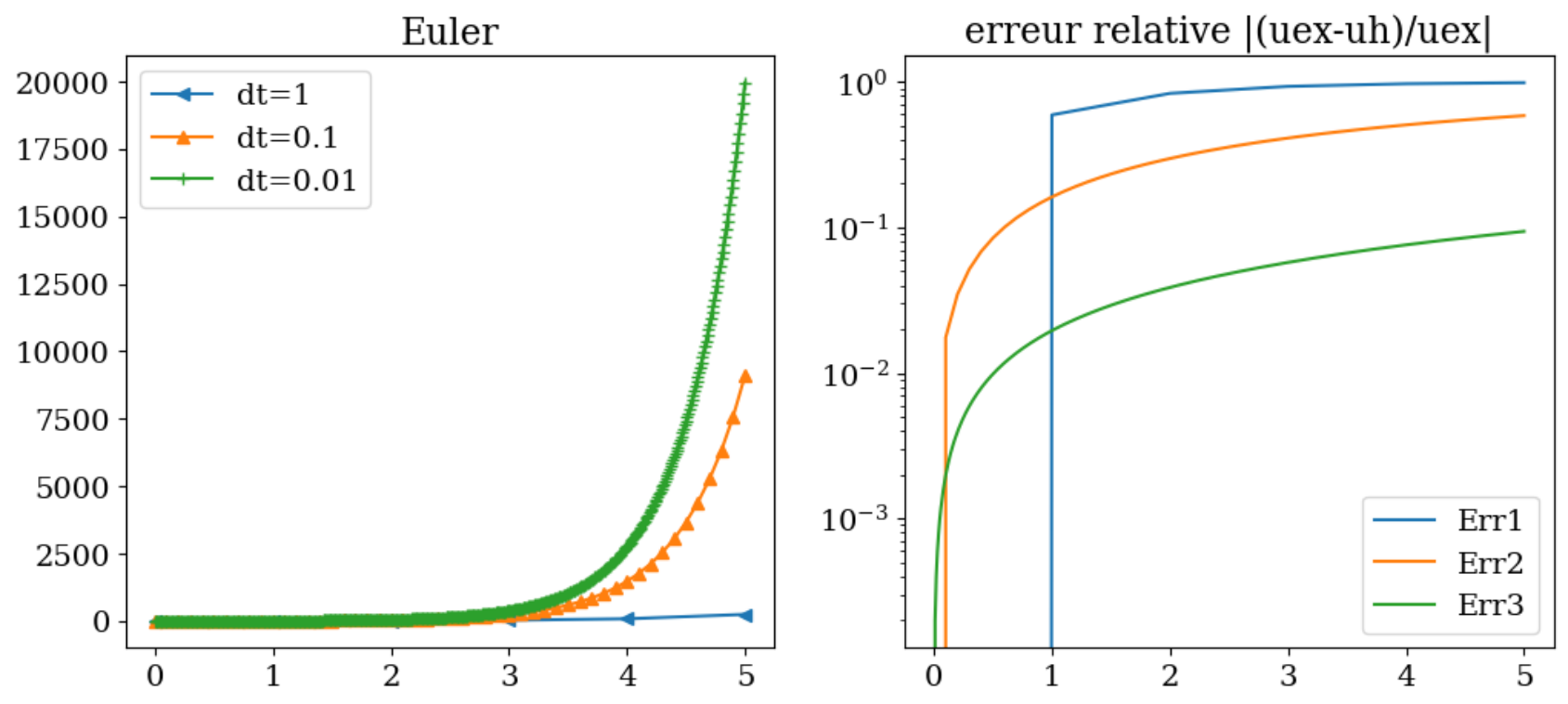
- Résolution avec différents pas dt
- Étude de l'évolution de l'erreur

Cas testés :

- $a = 0.2$ (cas facile)
- $a = 2.0$ (cas difficile)

```
a = 0.2  # cas facile
a = 2.0  # cas difficile
tf = 5
y0 = 1.0
# solution par Euler
Y1, T1 = Euler(F, y0, tf, 1.)
Y2, T2 = Euler(F, y0, tf, 0.1)
Y3, T3 = Euler(F, y0, tf, 0.01)
tracesol(T1, Y1, T2, Y2, T3, Y3, 'Euler')
```

Résultat numérique



On constate que la méthode d'Euler est une méthode peu précise avec une accumulation d'erreur au cours du temps, dû à sa précision d'ordre 1. La décroissance de l'erreur en fonction de dt est bien d'ordre 1 (sur la figure de droite, décroissance d'un facteur 10^1 pour un dt divisé par 10)

Exemple 2 : Utilisation de la méthode d'Euler

Considérons l'équation différentielle du premier ordre relative à la charge d'un condensateur dans un circuit RC :

$$\begin{cases} \frac{dU}{dt} + \frac{U}{\tau} = \frac{E}{\tau} \\ U(0) = 0 \end{cases}$$

où :

- $\tau = RC$ est la constante de temps du circuit,
- $U(t)$ est la tension aux bornes du condensateur,
- E est la tension de la source.

Paramètres numériques :

- Capacité : $C = 10^{-6}$ F
- Résistance : $R = 10^{3}$ Ω
- Tension source : $E = 6$ V
- $\tau = R \times C = 10^{-3}$ s
- Condition initiale : $U(0) = 0$

Méthode de Runge-Kutta d'ordre 2

Formule d'intégration :

$$y_{n+1} = y_n + \frac{dt}{2}(K_1 + K_2)$$

avec :

$$K_1 = f(t_n, y_n), \quad K_2 = f(t_n + dt,\, y_n + dt\, K_1)$$

Fonction générique RK2

```
# implementation m thode RK2
def RK2(f, y0, tf, dt):
    """Int gration de dy/dt = f(y, t) par Runge-Kutta
        2
       avec une CI y0 pendant tf avec un pas dt"""
    n = int(tf / dt)
    T = np.zeros(n + 1)
    Y = np.zeros(n + 1)
    Y[0] = y0
    T[0] = 0.0
    for i in range(n):
        K1 = f(Y[i], T[i])
        K2 = f(Y[i] + dt * K1, T[i] + dt)
        Y[i+1] = Y[i] + dt * (K1 + K2) / 2.
        T[i+1] = T[i] + dt
    return Y, T
```

Application : Résolution avec RK2

Équation :

$$\frac{dy}{dt} = ay(t) \quad \text{avec} \quad y(0) = 1$$

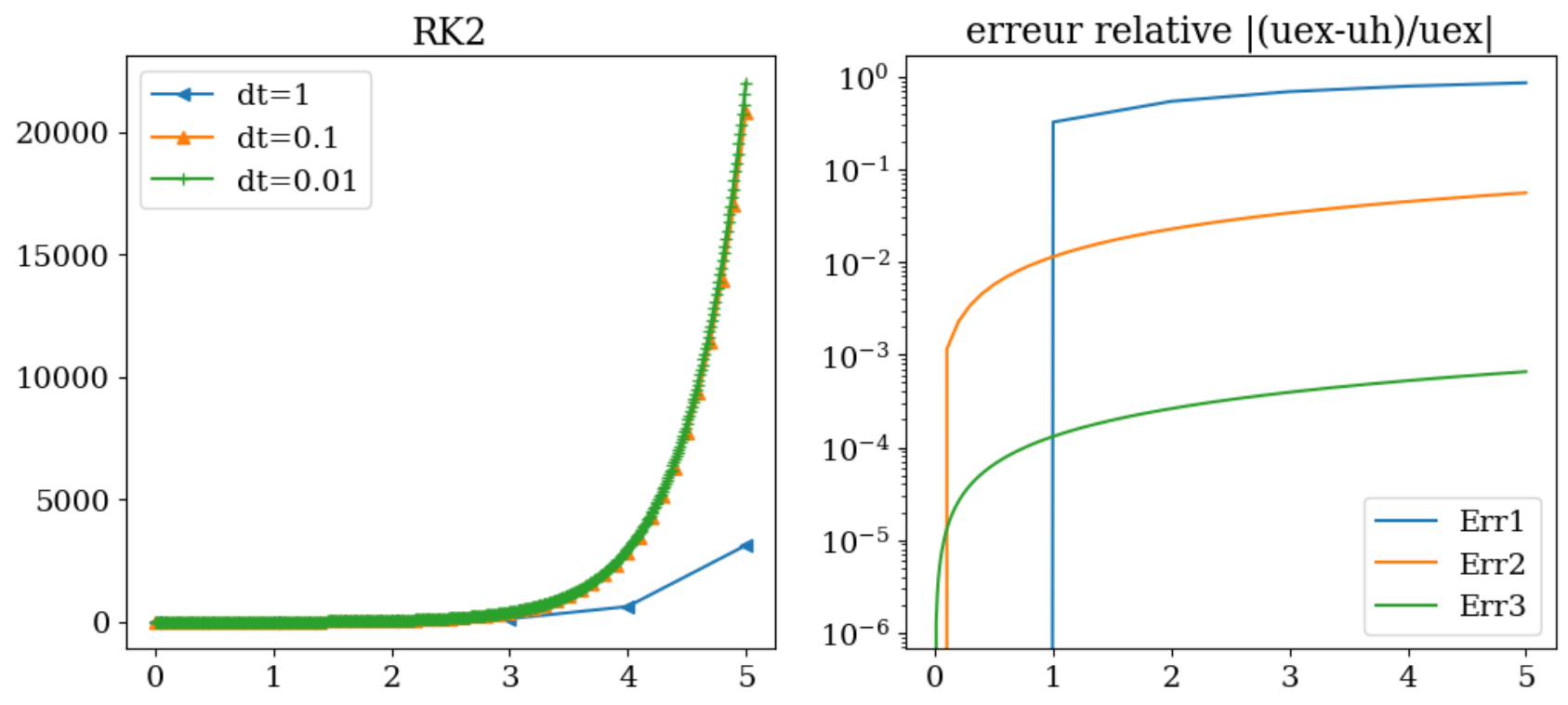
Objectif :

- Résolution avec la méthode de Runge-Kutta d'ordre 2 (RK2)
- Étude de l'erreur pour différents pas dt

```
# param tres
tf = 5
y0 = 1.0
# solution par RK2
Y1, T1 = RK2(F, y0, tf, 1.0)
Y2, T2 = RK2(F, y0, tf, 0.1)
Y3, T3 = RK2(F, y0, tf, 0.01)

# tracer solution et erreur
tracesol(T1, Y1, T2, Y2, T3, Y3, "RK2")
```

Résultat numérique



On constate que la solution est bien plus précise qu'avec la méthode d'Euler. La décroissance de l'erreur en fonction de dt est bien d'ordre 2 (sur la figure de droite, décroissance d'un facteur 10^2 pour un dt divisé par 10)

Méthode de Runge-Kutta d'ordre 4 (RK4)

Formule d'intégration :

$$y_{n+1} = y_n + \frac{dt}{6}(K_1 + 2K_2 + 2K_3 + K_4)$$

avec :

$$\begin{aligned}
K_1 &= f(t_n, y_n), \\
K_2 &= f\left(t_n + \frac{dt}{2},\, y_n + \frac{dt}{2} K_1\right), \\
K_3 &= f\left(t_n + \frac{dt}{2},\, y_n + \frac{dt}{2} K_2\right), \\
K_4 &= f(t_n + dt,\, y_n + dt\, K_3)
\end{aligned}$$

Fonction générique RK4

```
# implementation m thode RK4
def RK4(f, y0, tf, dt):

    n = int(tf / dt)
    T = np.zeros(n + 1)
    Y = np.zeros(n + 1)
    Y[0] = y0
    T[0] = 0.0
    for i in range(n):
        K1 = f(Y[i], T[i])
        K2 = f(Y[i] + dt/2 * K1, T[i] + dt/2)
        K3 = f(Y[i] + dt/2 * K2, T[i] + dt/2)
        K4 = f(Y[i] + dt * K3,   T[i] + dt)
        Y[i+1] = Y[i] + dt * (K1 + 2*K2 + 2*K3 + K4) /
             6
        T[i+1] = T[i] + dt
    return Y, T
```

Application : Résolution avec RK4

Équation :

$$\frac{dy}{dt} = ay(t) \quad \text{avec} \quad y(0) = 1$$

Objectif :

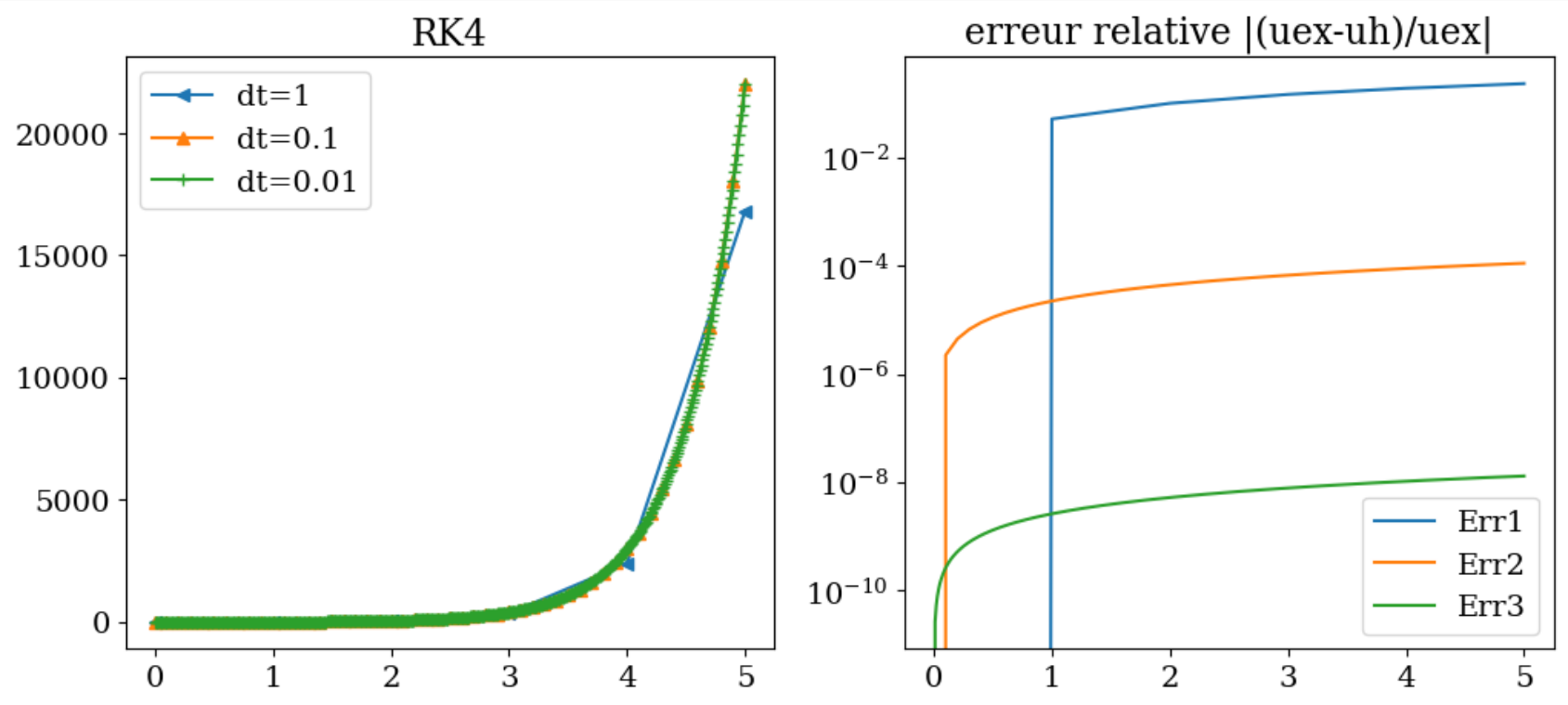
- Résolution avec la méthode de Runge-Kutta d'ordre 4 (RK4)
- Analyse de l'erreur en fonction du pas dt

```
# param tres
tf = 5
y0 = 1.0

# solution par RK4
Y1, T1 = RK4(F, y0, tf, 1.0)
Y2, T2 = RK4(F, y0, tf, 0.1)
Y3, T3 = RK4(F, y0, tf, 0.01)

# tracer solution et erreur
tracesol(T1, Y1, T2, Y2, T3, Y3, "RK4")
```

Résultat numérique



On constate que la solution est très précise mais avec de grand dt. La décroissance de l'erreur en fonction de dt est bien d'ordre 4 (sur la figure de droite, décroissance d'un facteur 10^4 pour un dt divisé par 10)

Méthode implicite : Schéma et linéarisation

Équation différentielle :

$$\frac{dy}{dt} = f(y,t)$$

Schéma implicite :

$$\frac{y^{n+1} - y^n}{\Delta t} = f(y^{n+1}, t^{n+1})$$

Problème non linéaire sur $\Delta y = y^{n+1} - y^n$ **:**

$$\frac{\Delta y}{\Delta t} - f(y^n + \Delta y, t^n + \Delta t) = 0$$

Linéarisation par développement limité (DL) à l'ordre 1 :

$$\frac{\Delta y}{\Delta t} - f(y^n, t^{n+1}) - \Delta y \frac{df}{dy} = 0$$

Extension possible : schéma implicite de Runge-Kutta (RK2)

Méthode de Runge-Kutta 2 implicite

Formule générale :

$$y_{n+1} = y_n + \frac{\Delta t}{2}\left[f(t_n, y_n) + f(t_{n+1}, y_{n+1})\right]$$

Problème implicite car y_{n+1} est présent dans les deux membres.

Linéarisation de $f(t_{n+1}, y_{n+1})$:

$$f(t_{n+1}, y_{n+1}) \approx f(t_{n+1}, y_n) + \frac{df}{dy} \cdot \Delta y$$

où $\Delta y = y_{n+1} - y_n$

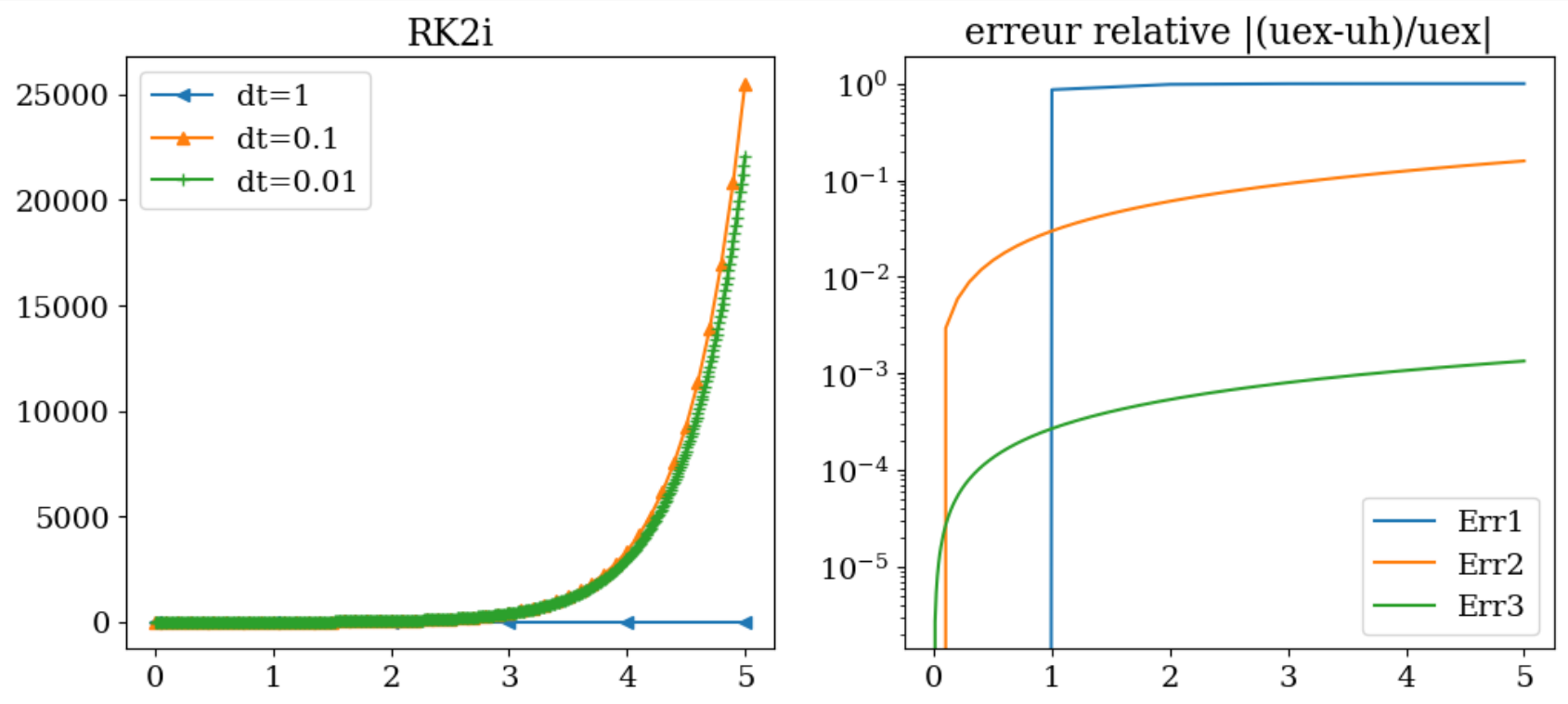
$$\Rightarrow \Delta y = \frac{f(t_n, y_n)}{1/\Delta t - \frac{df}{dy}}$$

Implémentation Python : RK2 Implicite

```python
def RK2i(f, y0, tf, dt):
    """Méthode Runge-Kutta 2 implicite pour dy/dt = f(y,t)"""
    n = int(tf/dt)
    T = np.zeros(n+1)
    Y = np.zeros(n+1)
    Y[0] = y0
    T[0] = 0.0
    for i in range(n):
        K1 = f(Y[i], T[i])
        DY = K1 / (1.0/dt - dF(Y[i], T[i]))  # linéarisation
        K2 = f(Y[i] + DY, T[i] + dt)
        Y[i+1] = Y[i] + (K1 + K2) * dt / 2.0
        T[i+1] = T[i] + dt
    return Y, T
```

Remarque : `dF(y,t)` retourne $\frac{df}{dy}$ évalué en (y, t).

Résultat numérique



La décroissance de l'erreur en fonction de dt est bien d'ordre 2 (sur la figure de droite, décroissance d'un facteur 10^2 pour un dt divisé par 10). Par contre, cette méthode n'apporte rien dans ce cas-là par rapport à une méthode RK2 explicite.

Méthodes intégrées dans `scipy.integrate.ode`

Bibliothèque `scipy.integrate.ode` :

- `dopri5` : Runge-Kutta d'ordre 4(5)
- `dopri853` : Runge-Kutta d'ordre 8(5,3)
- `vode` : méthodes implicites :
  - Adams pour problèmes non raides
  - BDF (Backward Differentiation Formula) pour problèmes raides
- `lsoda` : choisit automatiquement entre Adams et BDF

Avantage : pas de temps adaptatif pour atteindre une tolérance fixée.

Définition du second membre et du solveur

Exemple : $\frac{dy}{dt} = ay(t)$ avec $y(0) = 1$

Définition Python du second membre et de l'intégrateur :

```
from scipy import integrate
def solveODE(F, y0, tf, dt, methode):
    n = int(tf/dt)
    T = np.linspace(0, tf, n+1)
    Y = np.zeros(n+1)
    Y[0] = y0
    solver = integrate.ode(F, jac=None).set_integrator(methode)
    solver.set_initial_value(y0, T[0]).set_f_params(a)
    for i in range(1, n+1):
        if solver.successful():
            Y[i] = solver.integrate(T[i])
        else:
            print("Erreur ODE")
    return Y, T
```

Application : Méthode Runge-Kutta 4(5)

Équation test :

$$\frac{dy}{dt} = ay(t), \quad y(0) = 1$$

Méthode utilisée : `dopri5` (*Dormand-Prince* RK4(5))

Code Python utilisé :

```
tf = 5
y0 = 1.0
Y1, T1 = solveODE(FF, y0, tf, 1.0,  'dopri5')
Y2, T2 = solveODE(FF, y0, tf, 0.1,  'dopri5')
Y3, T3 = solveODE(FF, y0, tf, 0.01, 'dopri5')
tracesol(T1, Y1, T2, Y2, T3, Y3, "RK5")
```

Résultat : méthode RK5 avec différents Δt

- Comparaison des solutions numériques avec la solution exacte $y(t) = e^{at}$
- Tracé de l'erreur relative : $\left| \frac{y_{\text{num}} - y_{\text{exact}}}{y_{\text{exact}}} \right|$
- Test pour $a = 0.2$ (non raide) ou $a = 2.0$ (raide)

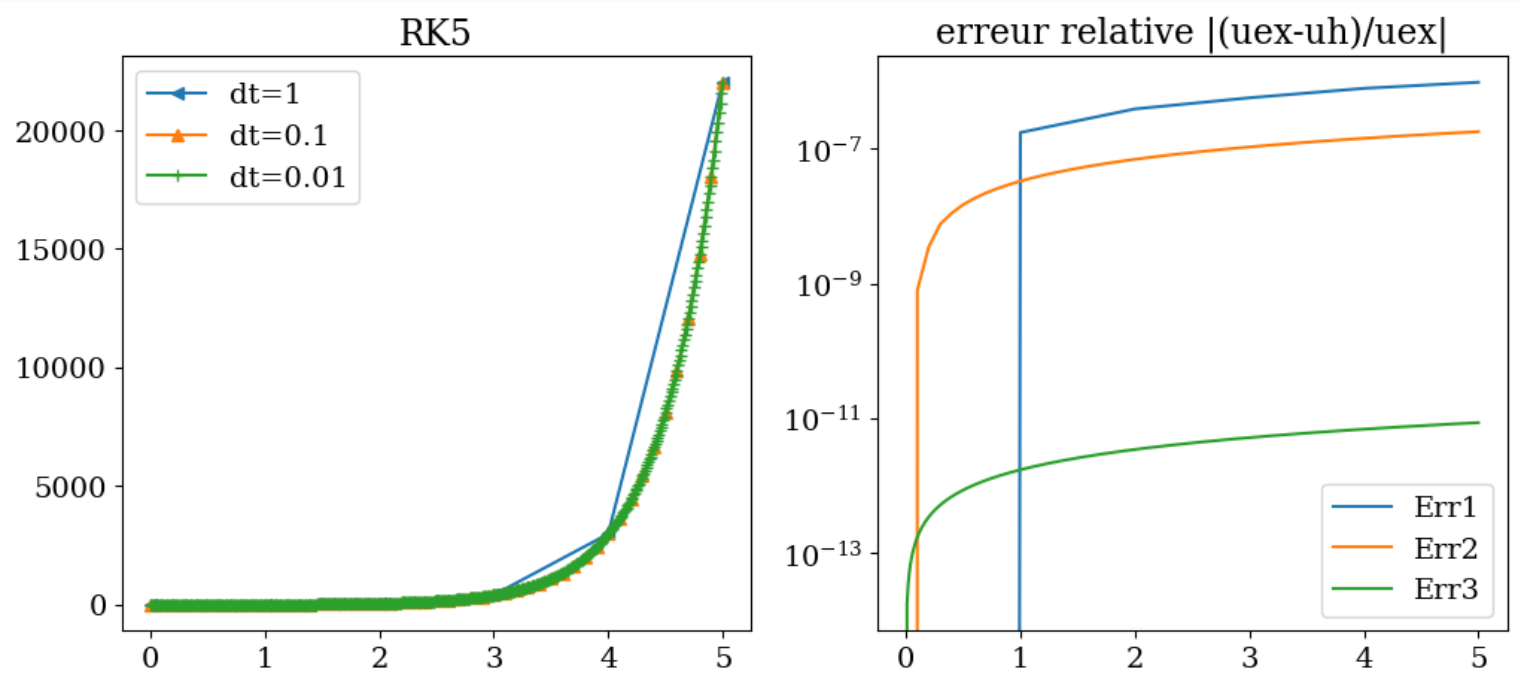


Figure 1: Solution et erreur relative pour RK5 (dopri5)

Application : Méthode Runge-Kutta 8

Équation test :

$$\frac{dy}{dt} = ay(t), \quad y(0) = 1$$

Méthode utilisée : `dop853` (*Runge-Kutta d'ordre 8*)

Code Python utilisé :

```
Y1, T1 = solveODE(FF, y0, tf, 1.0,  'dop853')
Y2, T2 = solveODE(FF, y0, tf, 0.1,  'dop853')
Y3, T3 = solveODE(FF, y0, tf, 0.01, 'dop853')
tracesol(T1, Y1, T2, Y2, T3, Y3, "RK8")
```

Résultat : méthode RK8 avec différents Δt

- Très haute précision grâce à l'ordre élevé
- Comparaison avec la solution exacte $y(t) = e^{at}$
- Erreur relative tracée pour différents pas de temps

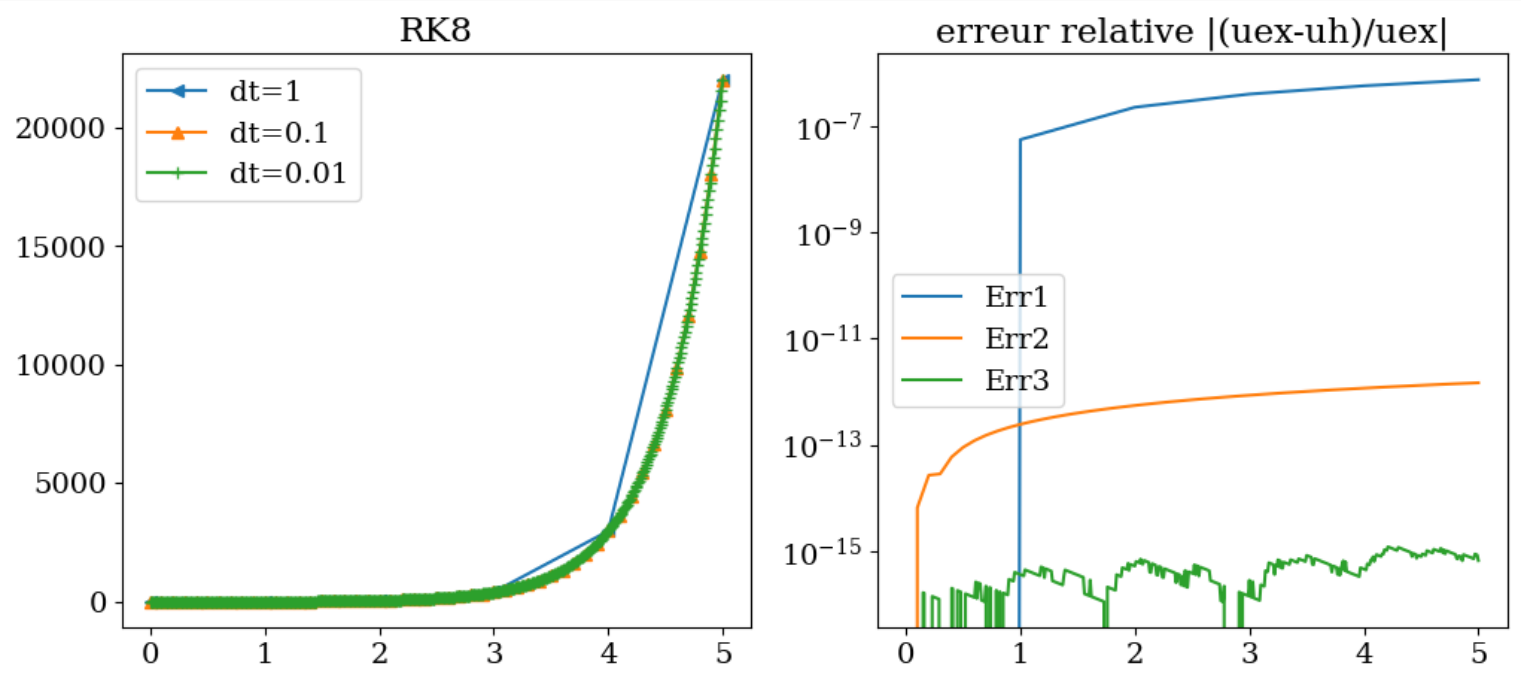


Figure 2: Solution et erreur relative pour RK8 (dop853)

Bibliothèque SciPy : `odeint`

Équation test :

$$\frac{dy}{dt} = ay(t), \quad y(0) = 1$$

Méthode utilisée : `odeint`

- Interface simple de `scipy.integrate`
- Méthode adaptative (LSODA) : Adams Bashford (non raide), BDF (raide)
- Choix automatique du schéma selon la raideur

Code Python :

```
from scipy.integrate import odeint
def F(y, t, a): return a*y

Y1 = odeint(F, y0, T1, args=(a,))
Y2 = odeint(F, y0, T2, args=(a,))
Y3 = odeint(F, y0, T3, args=(a,))
tracesol(T1,Y1[:,0],T2,Y2[:,0],T3,Y3[:,0],'odeint')
```

Résultat : méthode `odeint`

- Interface très simple
- Résolution automatique avec adaptation du schéma
- Haute précision même avec des pas larges si la fonction le permet

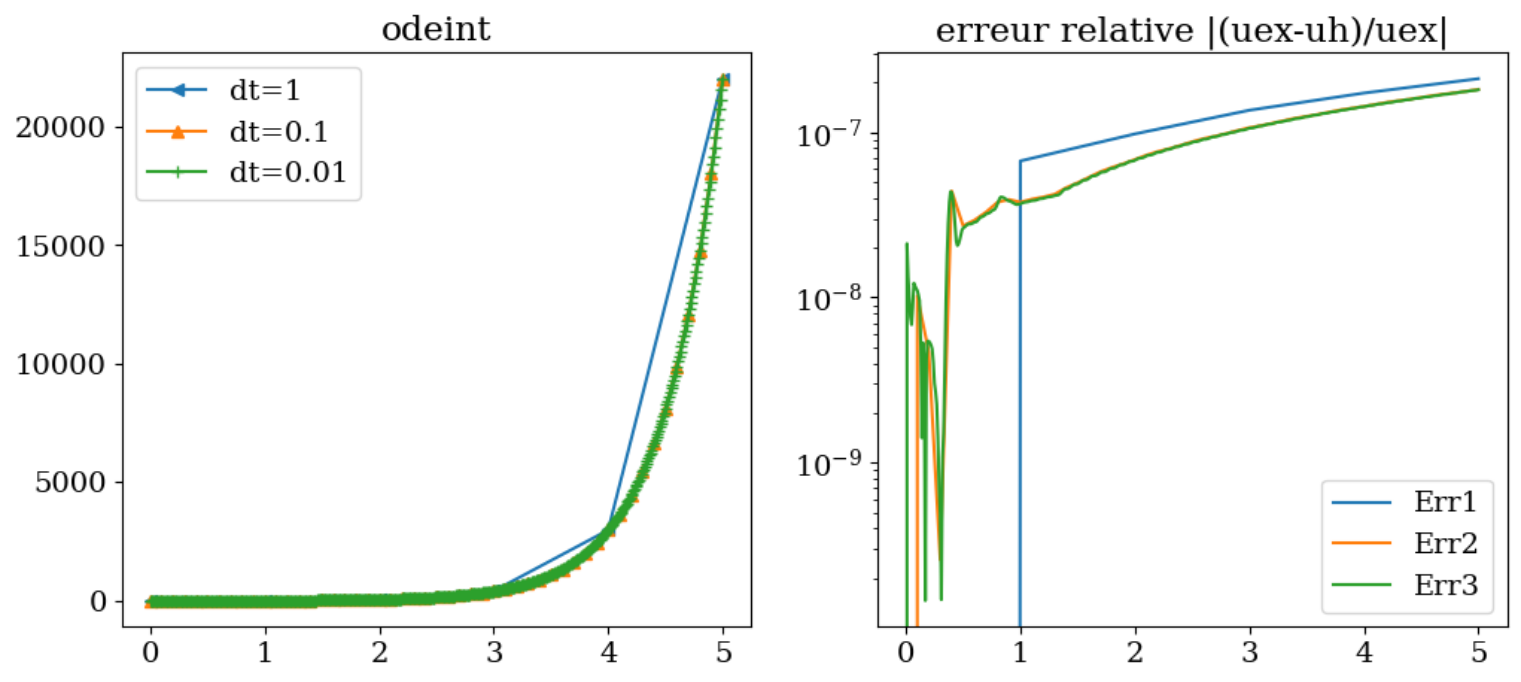


Figure 3: Solution et erreur relative pour `odeint`

On constate que **odeint** permet d'obtenir une erreur quasiment

Objectif : EDO d'ordre 2

- Résoudre une équation différentielle d'ordre 2 :

$$\frac{d^2y}{dt^2} = f(t, y, \frac{dy}{dt})$$

- Utiliser le **schéma d'Euler explicite**
- Montrer un exemple d'application numérique concret

Réduction en système d'ordre 1

Posons :

$$\begin{cases} y_1(t) = y(t) \\ y_2(t) = \dfrac{dy}{dt} \end{cases} \Rightarrow \begin{cases} \dfrac{dy_1}{dt} = y_2(t) \\ \dfrac{dy_2}{dt} = f(t, y_1, y_2) \end{cases}$$

Ce système sera résolu numériquement à l'aide du schéma d'Euler explicite.

Schéma d'Euler explicite

Pour un pas de temps h, le schéma devient :

$$\begin{cases} y_{1,n+1} = y_{1,n} + h \cdot y_{2,n} \\ y_{2,n+1} = y_{2,n} + h \cdot f(t_n, y_{1,n}, y_{2,n}) \end{cases}$$

Les valeurs y_1, y_2 sont connues à l'instant t_n, donc le schéma est **explicite**.

Exemple : Pendule simple

Soit un pendule de masse m et de longueur l, qui forme un angle $\theta(t)$ avec la verticale.

Formulation de Lagrange : $L = T - U$

$$T = \frac{1}{2} m l^2 \dot{\theta}^2, \quad U = -mgl\cos\theta, \quad \Rightarrow L = \frac{1}{2} m l^2 \dot{\theta}^2 + mgl\cos\theta$$

Équation de Lagrange :

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}}\right) - \frac{\partial L}{\partial \theta} = 0 \Rightarrow ml^2\ddot{\theta} + mgl\sin\theta = 0$$

$$\Rightarrow \ddot{\theta} + \frac{g}{l}\sin\theta = 0$$

Oscillateur harmonique : approximation linéaire

Dans le cas des petites oscillations : $\theta \ll 1$, on a $\sin\theta \approx \theta$

$$\Rightarrow \ddot{\theta} + \frac{g}{l}\theta = 0 \quad \text{avec } y = \theta, \quad \omega = \sqrt{\frac{g}{l}}$$

$$\Rightarrow \ddot{y} + \omega^2 y = 0$$

Conditions initiales :

$$y(0) = y_0, \quad \dot{y}(0) = \dot{y}_0$$

$$Y(t) = \begin{bmatrix} y \\ \dot{y} \end{bmatrix}, \quad F(Y,t) = \begin{bmatrix} Y[1] \\ -\omega^2 Y[0] \end{bmatrix}$$

$$Y(0) = \begin{bmatrix} \pi/10 \\ 0 \end{bmatrix}$$

Solution analytique de l'oscillateur harmonique

L'équation différentielle :

$$\ddot{y} + \omega^2 y = 0$$

a pour solution générale :

$$y(t) = y_0 \cos(\omega t) + \frac{\dot{y_0}}{\omega} \sin(\omega t)$$

Dans le cas :

$$y_0 = \frac{\pi}{10}, \quad \dot{y_0} = 0$$

$$\Rightarrow y(t) = \frac{\pi}{10} \cos(\omega t)$$

Cette solution est une oscillation harmonique de pulsation $\omega = \sqrt{\frac{g}{l}}$

Code python

```
# oscillateur harmonique
def Fl(Y,t,omega):
    """second membre linearise EDO"""
    return np.array([Y[1],-omega**2*Y[0]])
# cas non lineaire
def F(Y,t,omega):
    """second membre NL"""
    return np.array([Y[1],-omega**2*np.sin(Y[0])])
# parametres
L=1.0
g=10.0
omega=np.sqrt(g/L)
# integration sur 5 périodes
tmax = 5.*2*np.pi/omega
t  = np.linspace(0,tmax,100)
Y0 = np.array([np.pi/5,0.])
Yl = odeint(Fl,Y0,t,args=(omega,))
Y  = odeint(F,Y0,t,args=(omega,))
thetal=Yl[:,0]
theta =Y[:,0]
plt.plot(t,thetal,label="lineaire")
plt.plot(t,theta ,label="NL")
plt.legend()
plt.title("comparaison linéaire/non linéaire")
```

Résultats

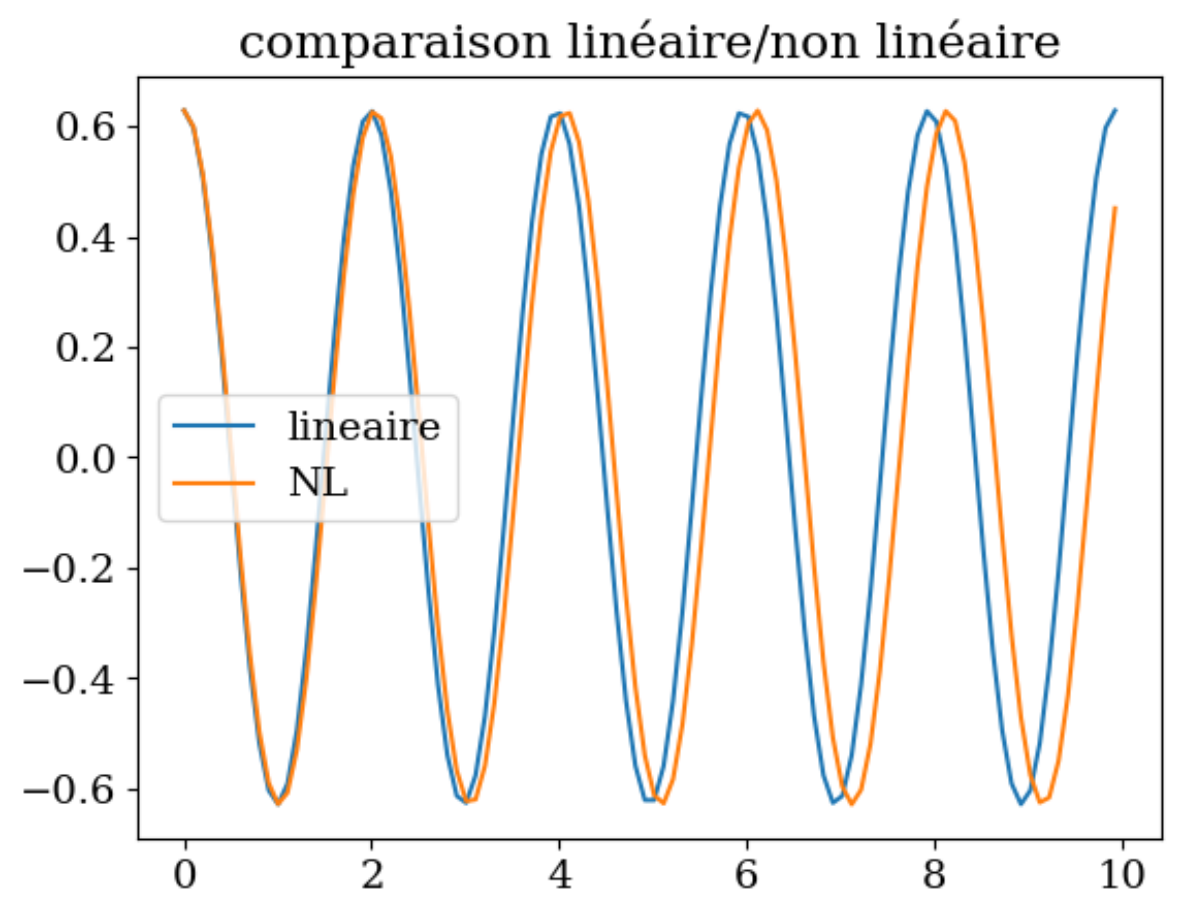


Figure 4: Comparaison linéaire et non linéaire

Exemple : Oscillateur harmonique amorti

Équation :

$$\frac{d^2y}{dt^2} + 2\zeta\omega\frac{dy}{dt} + \omega^2 y = 0$$

Réduction :

$$f(t, y_1, y_2) = -2\zeta\omega y_2 - \omega^2 y_1$$

Schéma d'Euler explicite :

$$\begin{cases} y_{1,n+1} = y_{1,n} + h \cdot y_{2,n} \\ y_{2,n+1} = y_{2,n} - h \cdot (2\zeta\omega y_{2,n} + \omega^2 y_{1,n}) \end{cases}$$

Application numérique

Paramètres :

- $\zeta = 0.1$, $\omega = 2\pi \approx 6.2832$
- $h = 0.01$, $T = 1\,\mathrm{s} \Rightarrow N = 100$
- Conditions initiales : $y_1(0) = 1$, $y_2(0) = 0$

Première itération calculée à la main :

$$
\begin{aligned}
y_{1,1} &= y_{1,0} + h \cdot y_{2,0} = 1 + 0.01 \times 0 = 1 \\
y_{2,1} &= y_{2,0} - h \cdot (2 \times 0.1 \times 6.2832 \times 0 + 6.2832^2 \times 1) \\
&= 0 - 0.01 \times 39.4784 \approx -0.3948
\end{aligned}
$$

Code Python pour la simulation complète :

```
import numpy as np
import matplotlib.pyplot as plt
zeta, omega = 0.1, 2 * np.pi
h, T = 0.01, 1.0
N = int(T / h)
t = np.linspace(0, T, N+1)
y1 = np.zeros(N+1)
y2 = np.zeros(N+1)
# Conditions initiales
y1[0] = 1.0
y2[0] = 0.0
for n in range(N):
    y1[n+1] = y1[n] + h * y2[n]
    y2[n+1] = y2[n] - h * (2 * zeta * omega * y2[n] + omega**2 * y1[n])
plt.plot(t, y1, label='Position $y(t)$')
plt.xlabel('Temps (s)')
plt.ylabel('y(t)')
plt.title('Oscillateur harmonique amorti - Euler explicite')
plt.grid(True)
plt.legend()
plt.show()
```

Résultats : évolution de la position y (t) et de la vitesse

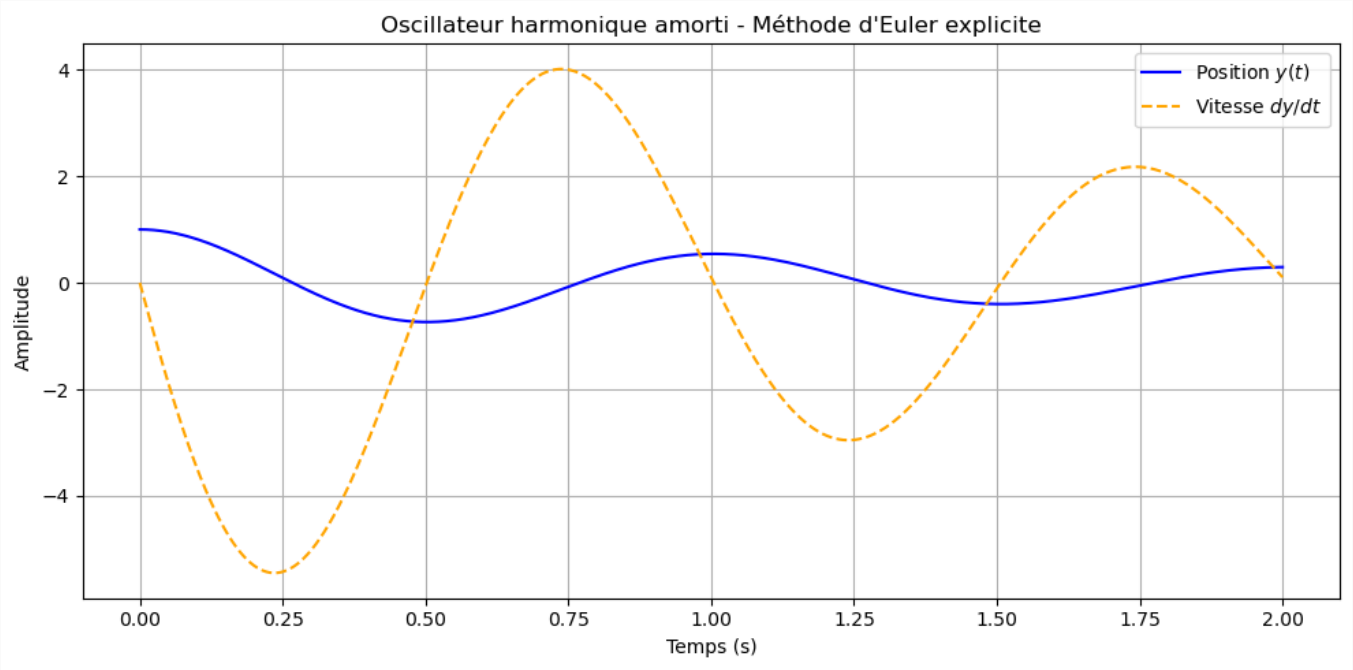


Figure 5: Oscillateur harmonique amorti

Remarques

- Le schéma est simple et rapide mais sensible au pas h
- Non-conservation de l'énergie, pouvant entraîner instabilités numériques
- Pour plus de stabilité, envisager schémas implicites ou d'ordre supérieur

Conclusion

- Euler explicite est un bon outil pédagogique
- Nécessite un choix adapté du pas de temps
- Étapes suivantes : méthodes implicites, Runge-Kutta, etc.